

README — data_vault/ (The Living LLM Wiki & Cognitive Memory Tiers)

Living Knowledge Graph, Karpathy LLM-Wiki Standard, Graphify AST & Multi-Tier Memory

Motto: *Xçineribus, in-variis-nunquam-varius, Novi-Æxentis-Copiæ, Vincent*

Scope: Definitive Technical Reference for the Living Wiki, OpenWiki CLI, Graphify AST Mapping, and the 5+1 Cognitive Memory Architecture.

1. Architectural Foundation: The Living LLM Wiki Standard

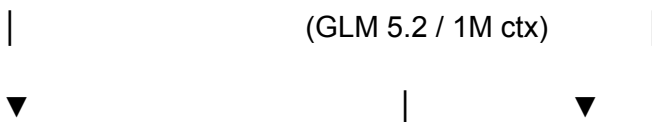
The **Data Vault** is not a passive folder of static notes. It implements **Andrej Karpathy's open-source LLM-Wiki pattern** combined with **LangChain's OpenWiki CLI**, **Graphify Labs AST parsing**, and **NotebookLM (notebooklm-py)**.

Instead of wasting prompt tokens by forcing frontier models to read entire raw codebases or documentation folders repeatedly, the system compiles all knowledge into a **queryable, compounding Markdown second brain** with bidirectional `[[wikilinks]]`.

[Raw Code & Docs] —(Graphify AST)—► [02_wiki_md/ast/]



[NotebookLM Research] —(notebooklm-py)—► [OPENWIKI CLI] —► [Obsidian Graph Vault]



[01_raw_sources/]

▼ Local Browsable Site

[03_recall_cache] (http://127.0.0.1:8765)

(Pre-tokenized)

2. The Four Pillars of the Living Wiki

Pillar 1: Graphify (Codebase & AST Dependency Mapper)

- **The Problem It Solves:** Raw code repositories contain complex multi-file dependencies. If an AI agent attempts to trace class inheritance or function calls across 100 files, it saturates its context window, loses track of execution chains, and incurs massive API costs.
- **The Implementation:**

```
uv tool run graphify . --obsidian --output 02_wiki_md/ast/
```

- **Technical Outcome:** Graphify parses the Abstract Syntax Tree (AST) of the entire codebase and exports atomic Markdown files for every class, function, and interface, complete with active `[[wikilinks]]`. This cuts agent token consumption by **up to 70%** during code review and planning passes.

Pillar 2: OpenWiki CLI & GLM 5.2 via OpenRouter

- **The Engine:** Runs LangChain's OpenWiki CLI natively inside Termux / Æsc.
- **Model Backend:** Powered by **Z.ai's GLM 5.2** via OpenRouter. With a **1,000,000 token context window**, GLM 5.2 ingests entire project directories and git diffs simultaneously for ~\$1.40 per million tokens.
- **Automated Git-Diff Maintenance Loop:** To prevent knowledge decay, every git commit triggers:

```
openwiki update
```

OpenWiki inspects only the modified git commits, re-runs Graphify strictly on changed files, updates the affected Obsidian notes, and maintains working bidirectional links without touching unaltered files.

Pillar 3: NotebookLM Analytical Research Pipeline (`notebooklm-py`)

- **The Integration:** Uses the unofficial `notebooklm-py` Python CLI wrapper.
- **Operational Role:** When heavy multi-document research, external audio transcripts, or technical whitepapers arrive, the agent triggers:

```
python3 tools/notebooklm/query_notebook.py --source 01_raw_sources/ --out  
02_wiki_md/concepts/
```

- It generates grounded cross-document summaries, deep conceptual FAQs, and synthesized knowledge blocks that feed directly into the living vault.

Pillar 4: Obsidian Local Vault & Flat-File Graph View

- **Vault Path:** `~/obsidian-vault-new`
- **Zero-Trust Boundary Compliance:** Operates purely on local flat `.md` files without touching hidden `.obsidian/` visual workspace configurations.
- **Graph View:** Renders visual topological relationship maps connecting code dependencies, NotebookLM research notes, and OpenWiki summaries.
- **Local Web Mirror (llm-wiki):** Compiles the vault into a static, high-speed HTML site served locally on `http://127.0.0.1:8765`:

```
python tools/llm-wiki/compiler.py --src 02_wiki_md/ --build dist/
```

```
python -m http.server 8765 --directory dist/
```

3. The 5+1 Cognitive Memory Tier Specification

data_vault/ (The Living Memory Engine)

|

— 📄 MAP.md	# Master human ontology & cross-tier link graph
— 📊 manifest.jsonl	# Machine-readable SHA256 catalog & entity index

|

— 📁 01_raw_sources/	# TIER 1: COLD SENSORY ARCHIVE (Read-Only)
---------------------	--

— manifest.jsonl	# Local raw asset provenance index
------------------	------------------------------------

— pdf/	# Research whitepapers & hardware specs (*.pdf)
--------	---

— media/	# Audio memos, VAD voice recordings (*.wav, *.mp4)
----------	--

— text/	# Raw dumps, API payloads, web scrapes (*.txt, *.html)
---------	--

|

— 📁 02_wiki_md/	# TIER 2: THE LIVING WIKI (OpenWiki TUI / Obsidian)
-----------------	---

— manifest.jsonl	# Graph node and wikilink catalog
— ast/	# Graphify AST dependency graphs (*.md)
— concepts/	# Atomic conceptual notes (Zettelkasten domain models)
— architectures/	# Technical specs & cross-repo blueprints
— entities/	# Device registers, schema contracts, hardware profiles
— indexes/	# Maps of Content (MOCs) & taxonomy hubs
— 📁 03_recall_cache/ (High-Speed Buffer)	# TIER 3: WORKING MEMORY ACCELERATOR
— manifest.jsonl	# Shard index
— jsonl/	# Pre-tokenized passages for instant prompt injection
— vectors/	# Dense embedding indices (FAISS / HNSW checkpoints)
— kv_store/	# Embedded SQLite key-value fast lookup tables
— 📁 04_skills_runtime/	# TIER 4: PROCEDURAL REPERTOIRE (Skills & Tools)
— manifest.jsonl	# Local procedural catalog
— prompt_skills/	# Modular SKILL.md definitions with YAML frontmatter
— extracted_tools/	# Deterministic code blocks (cli/*.sh, wrappers/*.py)
— runtimes/	# Daemon operational scripts & socket listeners
— policies/	# Self-correction heuristics & validation schemas

└─ 📁 05_episodic_logs/ Interaction Traces)	# TIER 5: EPISODIC TELEMETRY (Append-Only)
├─ manifest.jsonl	# Session & audit registry
├─ daily_driver_sync/	# Ingested logs from mobile edge sessions
├─ trajectories/	# Tri-model run traces (Query, Executor, Frontier)
├─ rlvr_verifiers/	# Graded rewards (+1.0 / -1.0) & assertion outcomes
├─ hygiene_reports/	# failure_log_template.md & schema integrity audits
└─ .incognito_red_sandbox/ quarantine	# [SHADOW ISOLATION] Sealed Red Auditor

4. Zero-Trust Housekeeping & Leak Prevention

To prevent unverified notes or agent hallucinations from corrupting the living wiki:

1. **Strict Frontmatter Invariant:** Every file in `02_wiki_md/` must include YAML frontmatter declaring `id`, `tier: 2`, `type`, `sources`, and `tags`.
2. **Context Leak Sweeper (`system_housekeeper.sh`):**
 - Continuously audits `~/obsidian-vault-new` against `master_blueprint.txt`.
 - If an unindexed Markdown note is detected, it flags: `[WARN] UNREGISTERED OBSIDIAN ASSET FOUND: <filename> (Context Leak Risk)`
 - Forces either human validation or automated OpenWiki registration before the note can be ingested by OB1 or downstream agents.